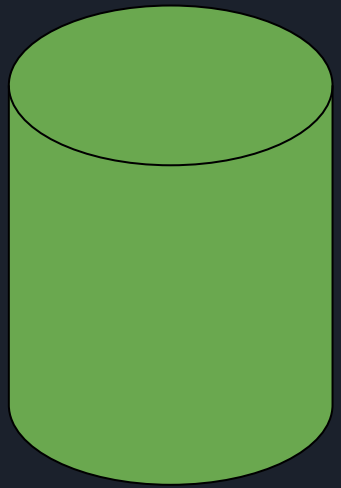
GESTIÓN DE DATOS

ADMINISTRACIÓN: MOTORES

RECORDERIS...



¿QUÉ APRENDEREMOS HOY?



- Qué es un motor de almacenamiento
- Tipos de motores por defecto en MariaDB
- Cuando usar cada uno
- "Recorderis":
  - Transacciones
  - Función "cast"
  - Función "rand"
  - Función "date_add"
  - Función "uuid" y "md5"

PARA REFLEXIONAR...

¿Usted ha formateado alguna vez un medio de almacenamiento?

¿Que sistema de archivos eligió y porqué?

(discusión verbal)



¿Qué podrá ser un "MOTOR"



Def: Motor de almacenamiento

Componentes que gestionan ***cómo se guardan los datos*** de la base de datos y cómo se accede a ellos.

Dos tablas iguales pueden comportarse distinto solo por tener motores de almacenamiento diferentes

analogía

Un disco guardará ***la misma*** información sin importar si está formateado en fat32, ext4, ntfs, etc. Lo que cambia es ***cómo*** la guarda. Lo mismo pasa con los motores en un dbms.

Los motores son a las tablas de los dbms como los sistemas de archivos son a un medio de almacenamiento.

Motores de almacenamiento estándar

Motor	Uso recomendado	Observaciones
InnoDB	Transaccional	Motor por defecto en MariaDB y MySQL. Buen manejo de concurrencia.
MyISAM	No transaccional	Motor clásico de MySQL. Optimizado para lecturas pero no para concurrencia
Memory	No transaccional	Velocidad extremadamente rápida. Recomendado para datos temporales o caché, y pequeños
CSV	No transaccional	No recomendado para el trabajo diario. No soporta campos opcionales

Motores de almacenamiento alternos

Motor	Uso recomendado	Observaciones
mrg_myisam	No transaccional	Permite ver varias tablas MyISAM (iguales en su estructura) como si fuera una sola
blackhole	Sólo pruebas	Sólo estructura. No maneja datos
archive	Archivado	Para tablas cuyos datos sean de poco uso

Motores sólo para MariaDB

Motor	Uso recomendado	Observaciones
aria	No transaccional	Evolución de MyISAM
federatedX	Acceso remoto	Para bases de datos MariaDB/MySQL remotas
connect	Acceso externo hacia archivos de diferentes formatos	Soporta archivos ini, csv, xml, json, conectores odbc, jdbc y algunas bases de datos nosql
spider	Acceso remoto	Ver una tabla remota como si fuera local
sequence	Generación de datos para pruebas	No es necesario crear tablas (Se darán ejemplos más adelante)

Comparación motores más relevantes

Motor	Observaciones
MyISAM	Rápido para leer, lento para escribir. No transaccional. Bloqueos de tabla completa en CRUD. Baja tolerancia a fallos. Muy en uso para sistemas legados.
Aria	Muy rápido para leer, muy lento para escribir. No transaccional. Bloqueos de tabla parcial en CRUD. Mejor tolerancia a fallos que MyISAM. Solo MariaDB (no MySQL)
InnoDB	Rápido para leer, rápido para escribir. Transaccional. Buena tolerancia a fallos. Bloqueos a nivel de registro (no de tabla). Es el más usado y relevante de todos los motores

Motores sólo para MariaDB

Además de los anteriores hay muuuuuuuuuuuuuuuuuuuchos más...

Referencias oficiales:

MySQL:
https://dev.mysql.com/doc/refman/8.3/en/innodb-storage-engine.html
https://dev.mysql.com/doc/refman/8.3/en/storage-engines.html

MariaDB:
https://mariadb.com/kb/en/storage-engines/

¿Cual motor debo elegir?

Respuesta corta: Depende de la necesidad

Respuesta larga: Se debe conocer la transaccionalidad, cantidad de operaciones de lectura y escritura, crecimiento de la tabla en registros / tiempo, medio de almacenamiento físico a usar, cantidad de usuarios que accederán a la tabla, y un largo etcétera...

¿Cual motor debo elegir?

No hay una receta mágica. Pero en general se puede decir que:

- tablas transaccionales: InnoDB
- tablas de lectura intensiva y poca escritura: Aria* o MyISAM
- caché o datos temporales pequeños: Memory
- ahorro de espacio para datos archivados: Archive
- conexión a fuentes externas: Connect*
- no hay datos para decidir o no se sabe cual escoger: InnoDB

* Sólo MariaDB, no MySQL

¿Cual motor debo elegir?

Notas importantes a tener en cuenta:

- **No es lo mismo un disco mecánico que un disco de estado sólido. Tanto por velocidad como por vida útil**
  - **Mito: un SSD siempre mejora el rendimiento.**
  - **Realidad: un SSD mal aprovechado sigue siendo lento y mucho menos soluciona problemas de diseño**

- **La latencia de red es importante. Importante si se deciden usar motores que se conectan a bases de datos externas**

¿Cual motor debo elegir?

Notas importantes a tener en cuenta:

- **¡La memoria no es infinita!** Hay que usarla correctamente en los motores "memory". Este motor es rapidísimo... hasta que colapsa.

- "Archive" ahorra espacio pero sacrifica cpu. "Connect" es muy versátil pero implica cpu para las transformaciones. Siempre dependerá del caso puntual cuando usar uno u otro.

Consejo de diseño y arquitectura

Los motores "connect", "federated", "spider" existen y son soportados por MariaDB (¡y por algo será!)

Usarlos sólo cuando sea estrictamente necesario. Realizar conexiones directas entre capas de datos no es una buena idea de arquitectura (...)

En sistemas tradicionales, es más que suficiente con:

- InnoDB + Aria (MariaDB)
- InnoDB + MyISAM (MySQL)
- InnoDB

CONSULTA

Estimado estudiante:

Tómate el trabajo de investigar por qué realizar conexiones directas entre capas de datos no es una buena idea de arquitectura ni de diseño

Motores de almacenamiento: archivos

Motor	Archivos generados
InnoDB	`*.frm (estructura), *.ibd (datos)`
MyISAM	`*.frm (estructura), *.MYD (datos), *.MYI (índices)`
Memory	`*.frm (estructura)`
CSV	`*.frm (estructura), *.CSV (datos), *.CSM (metadatos)`
mrg_myisam	`*.frm (estructura), *.MRG (metadatos)`
blackhole	`*.sdi (estructura)`

Motores de almacenamiento: archivos

Motor	Archivos generados
archive	*.sdi (estructura), *.ARZ (datos)
aria	*.frm (estructura), *.MAD (datos), *.MAI (índices)
federatedX	*.frm (estructura)
connect	Depende de los archivos externos a usar
sequence	Ninguno

Para un DBA esta información es clave para diagnosticar errores y hacer respaldos

¿Donde quedan los archivos por defecto?

Windows:

C:/Program Files/MariaDB 10.X/data
C:/Program Files/MySQL/data

***nix (linux, mac, etc...):**

/var/lib/mysql/

Acá viven tus datos. Si estas rutas se dañan... recuerda que en la vida hay 4 cosas que no regresan...

Motor SEQUENCE

Es un motor especial que incluye MariaDB con el objetivo de generar secuencias con valores *positivos* (tipo de dato: bigint unsigned) de inicio, fin y saltos.

Así se usa:

seq_x_to_y	`Secuencia ascendente desde x hasta y`
seq_x_to_y_step_z	`Secuencia ascendente desde x hasta y, con saltos de z`
seq_y_to_x	`Secuencia descendente desde y hasta x`
seq_y_to_z_step_z	`Secuencia descendente desde y hasta x, con saltos de z`

Motor SEQUENCE: Ejemplos

Secuencia de 1 a 10

```
MariaDB [gd]> select * from seq_1_to_10;
+-----+
| seq |
+-----+
|   1 |
|   2 |
|   3 |
|   4 |
|   5 |
|   6 |
|   7 |
|   8 |
|   9 |
|  10 |
+-----+
10 rows in set (0,001 sec)
```

Secuencia de 1 a 10, de 2 en 2

```
MariaDB [gd]> select * from seq_1_to_10_step_2;
+-----+
| seq |
+-----+
|   1 |
|   3 |
|   5 |
|   7 |
|   9 |
+-----+
5 rows in set (0,003 sec)
```

Motor SEQUENCE: Ejemplos

Secuencia de 5 a 1

```
MariaDB [gd]> select * from seq_5_to_1;
+-----+
| seq |
+-----+
|   5 |
|   4 |
|   3 |
|   2 |
|   1 |
+-----+
5 rows in set (0,000 sec)
```

Secuencia de 10 a 1, de 3 en 3

```
MariaDB [gd]> select * from seq_10_to_1_step_3;
+-----+
| seq |
+-----+
|  10 |
|   7 |
|   4 |
|   1 |
+-----+
4 rows in set (0,038 sec)
```

Motor SEQUENCE

Si necesitas generar datos para pruebas y simulaciones usando sólo SQL... para eso será el motor sequence

Recorderis: Función CAST

Es una función que permite convertir un tipo de dato a otro.

```
MariaDB [gd]> select cast('123' as int) as value;
+-------+
| value |
+-------+
|   123 |
+-------+
1 row in set (0,011 sec)
```

Varchar a entero

```
MariaDB [gd]> select cast(-1 as unsigned int) as value;
+----------------------+
| value                |
+----------------------+
| 18446744073709551615 |
+----------------------+
1 row in set, 1 warning (0,000 sec)
```

Entero a entero sin signo (con warnings)

Motor SEQUENCE: Ejemplos

Como el tipo de dato es bigint unsigned, no está permitido retornar negativos

```
MariaDB [gd]> select (seq)-10 from seq_1_to_5;
ERROR 1690 (22003): BIGINT UNSIGNED value is out of range in '`gd`.`seq_1_to_5`.`seq` - 10'
```

¿Solución?
Convertir la secuencia de "unsigned bigint" a "bigint"



```
MariaDB [gd]> select cast(seq as int) -10 as value from seq_9_to_0;
+-------+
| value |
+-------+
|    -1 |
|    -2 |
|    -3 |
|    -4 |
|    -5 |
|    -6 |
|    -7 |
|    -8 |
|    -9 |
|   -10 |
+-------+
10 rows in set (0,005 sec)
```

Generación de datos para pruebas

Generación de fechas



```
MariaDB [gd]> select date_add('2030-01-10', interval seq day) as value from seq_0_to_40;
+------------+
| value      |
+------------+
| 2030-01-10 |
| 2030-01-11 |
| 2030-01-12 |
| 2030-01-13 |
| 2030-01-14 |
| 2030-01-15 |
| 2030-01-16 |
| 2030-01-17 |
| 2030-01-18 |
| 2030-01-19 |
| 2030-01-20 |
| 2030-01-21 |
| 2030-01-22 |
| 2030-01-23 |
| 2030-01-24 |
| 2030-01-25 |
| 2030-01-26 |
| 2030-01-27 |
| 2030-01-28 |
| 2030-01-29 |
| 2030-01-30 |
| 2030-01-31 |
| 2030-02-01 |
| 2030-02-02 |
| 2030-02-03 |
```

Generación de datos para pruebas

EJERCICIO:

Generar mediciones de temperatura aleatorias entre 15 y 25 grados para pruebas, para el mes de febrero del año 2045.

SOLUCIÓN:

Usar "seq" y "rand".

```
MariaDB [gd]> select date_add('2045-02-01', interval seq day) as my_date,
    -> round((15+rand()*10),1) as temperature
    -> from seq_0_to_27;
+------------+-------------+
| my_date    | temperature |
+------------+-------------+
| 2045-02-01 |        25.0 |
| 2045-02-02 |        17.6 |
| 2045-02-03 |        18.1 |
| 2045-02-04 |        22.5 |
| 2045-02-05 |        23.2 |
| 2045-02-06 |        23.7 |
| 2045-02-07 |        23.7 |
| 2045-02-08 |        22.5 |
| 2045-02-09 |        16.5 |
| 2045-02-10 |        20.1 |
| 2045-02-11 |        15.8 |
| 2045-02-12 |        23.8 |
| 2045-02-13 |        16.4 |
| 2045-02-14 |        15.9 |
| 2045-02-15 |        25.0 |
| 2045-02-16 |        22.3 |
| 2045-02-17 |        21.4 |
| 2045-02-18 |        15.3 |
| 2045-02-19 |        17.1 |
| 2045-02-20 |        24.7 |
| 2045-02-21 |        17.3 |
| 2045-02-22 |        17.2 |
| 2045-02-23 |        19.3 |
| 2045-02-24 |        19.9 |
| 2045-02-25 |        16.3 |
| 2045-02-26 |        17.2 |
| 2045-02-27 |        21.7 |
| 2045-02-28 |        22.3 |
```

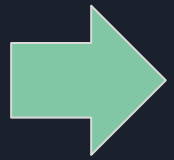
Generación de datos para pruebas

EJERCICIO:

Igual que el anterior, más un texto aleatorio.

SOLUCIÓN:

Usar "seq", "rand", "md5" y "uuid".



```
MariaDB [gd]> select date_add('2045-02-01', interval seq day) as my_date,
    -> round((15+rand()*10),1) as temperature,
    -> md5(uuid()) as some_text
    -> from seq_0_to_27;
+------------+-------------+----------------------------------+
| my_date    | temperature | some_text                        |
+------------+-------------+----------------------------------+
| 2045-02-01 |        18.2 | 2694817a555b14dad4c79041975a5111 |
| 2045-02-02 |        18.5 | 43cc95611a611f08d2b29d5435513e09 |
| 2045-02-03 |        23.2 | ae57b88e0fa2f77cc88d8df2c9b1641e |
| 2045-02-04 |        15.3 | 162d3654a93c594b9bd0b5934b13f2ed |
| 2045-02-05 |        21.9 | da8d1fc40552bb2379871abdc72be87d |
| 2045-02-06 |        18.8 | 1435b0d79e256c6222b6316ae7530087 |
| 2045-02-07 |        23.4 | 0c281ed1154860a1aff255f699356830 |
| 2045-02-08 |        15.4 | af470e002584dd6bc37340cdc5973f14 |
| 2045-02-09 |        21.7 | 03d4308e94b42208ea48a9e7b9619cdb |
| 2045-02-10 |        17.3 | 19aeba245e26494e1380d28c91827232 |
| 2045-02-11 |        16.4 | 3f6bb02adb214d94964052d83fb59e41 |
| 2045-02-12 |        15.4 | d0331d2be2c84147a7ea6f94f8ff1064 |
| 2045-02-13 |        22.4 | 3c64138fdbed92510029a1d75de0175e |
| 2045-02-14 |        21.1 | bd957884c31ac2f26ed7bd74b5550d9c |
| 2045-02-15 |        23.2 | 39d30fe354d19c2772f57b9515770e4d |
| 2045-02-16 |        17.7 | 82f45f950e5d67eb3367564ed9ddc6ab |
| 2045-02-17 |        24.0 | deff9b1bdb6d707c4d04466ba0bfa884 |
| 2045-02-18 |        21.8 | 2827139c7594b637896846246266c699 |
| 2045-02-19 |        22.2 | 3c742df2ead9759097265ac2986339d9 |
| 2045-02-20 |        20.3 | 16a15d6bb9fc9ab8597772020a0002ef |
| 2045-02-21 |        20.1 | 6bba573bf390221269749f06ce73316f |
| 2045-02-22 |        24.6 | 4649188b3db3fcf60cc73f0fd9c52cae |
| 2045-02-23 |        17.5 | 958153fe85c08b4c347faed06b74e5ed |
| 2045-02-24 |        18.8 | bfaae228bd21791f6663ea14d0f33deb |
| 2045-02-25 |        16.6 | a45a9aeee6cd645f08462b0ede29b424 |
| 2045-02-26 |        21.6 | f3bca5454101401ed05a3f678428bdef |
| 2045-02-27 |        23.3 | 0870e3cb94c452a83fdd6ed3549a5197 |
| 2045-02-28 |        16.6 | adcb9723d15b95173380ba7d170255e3 |
```

¿Que motores tiene mi instancia?

```
MariaDB [gd]> show engines;
+--------------------+---------+------------------------------------------------------------------------------------------------+--------------+-----
-+------------+
| Engine             | Support | Comment                                                                                        | Transactions | XA
 | Savepoints |
+--------------------+---------+------------------------------------------------------------------------------------------------+--------------+-----
-+------------+
| CSV                | YES     | Stores tables as CSV files                                                                     | NO           | NO
 | NO         |
| MRG_MyISAM         | YES     | Collection of identical MyISAM tables                                                          | NO           | NO
 | NO         |
| MEMORY             | YES     | Hash based, stored in memory, useful for temporary tables                                      | NO           | NO
 | NO         |
| Aria               | YES     | Crash-safe tables with MyISAM heritage. Used for internal temporary tables and privilege tables | NO           | NO
 | NO         |
| MyISAM             | YES     | Non-transactional engine with good performance and small data footprint                        | NO           | NO
 | NO         |
| SEQUENCE           | YES     | Generated tables filled with sequential values                                                 | YES          | NO
 | YES        |
| InnoDB             | DEFAULT | Supports transactions, row-level locking, foreign keys and encryption for tables               | YES          | YES
 | YES        |
| PERFORMANCE_SCHEMA | YES     | Performance Schema                                                                             | NO           | NO
 | NO         |
+--------------------+---------+------------------------------------------------------------------------------------------------+--------------+-----
-+------------+
8 rows in set (0,123 sec)
```

¿Cómo especificar el motor?

```
MariaDB [gd]> create table table1(id int) engine Aria;
Query OK, 0 rows affected (0,178 sec)

MariaDB [gd]> create table table2(id int) engine MyISAM;
Query OK, 0 rows affected (0,067 sec)

MariaDB [gd]> create table table3(id int) engine Memory;
Query OK, 0 rows affected (0,006 sec)

MariaDB [gd]> create table table4(id int) engine InnoDB;
Query OK, 0 rows affected (0,431 sec)

MariaDB [gd]> create table table5(id int) engine CSV;
ERROR 1178 (42000): The storage engine for the table doesn't support nullable columns
MariaDB [gd]> create table table5(id int not null) engine CSV;
Query OK, 0 rows affected (0,042 sec)
```

EJERCICIOS

Ejercicio 1

Crear 6 tablas sobre una base de datos nueva (la puede llamar como usted prefiera). Los nombres de las tablas serán "t_aria", "t_myisam", "t_innodb", "t_memory", "t_csv", "t_blackhole". La estructura de estas tablas debe ser la siguiente (igual para todas las tablas). Observar que no hay clave primaria:

- my_id, int(11), obligatorio
- my_date, date, obligatorio
- my_value, real, opcional
- my_description, varchar(50), opcional

EJERCICIOS

NOTA IMPORTANTE

Los siguiente ejercicios implican escribir, y posteriormente ejecutar procedimientos almacenados.

Dado que estos manejarán un gran volumen de datos, asegurarse de iniciar una transacción antes de ejecutar el procedimiento y confirmarla una vez la ejecución termine. De lo contrario la ejecución tardará horas en terminar.

Como "recorderis", las palabras clave son "start transaction", "commit", "rollback".

EJERCICIOS

Ejercicio 2

Crear un procedimiento almacenado llamado "pr_fill", con parámetros de entrada p_table_name, varchar(30), p_records unsigned bigint.

El procedimiento debe insertar en la tabla p_table_name la cantidad de registros especificados en la variable p_records con los siguientes valores:

- my_id: número de iteración actual
- my_date: Fecha generada a partir de una secuencia, años entre 2000 y 2050
- my_value: Número real aleatorio entre 0 y 9999
- my_value_type: enum con posibles valores "ODD" y "EVEN"
- my_description: Texto de 32 caracteres, diferente para cada registro

EJERCICIOS

Ejercicio 2 (continuación)

- Puede apoyarse en las tablas con motor "sequence", las funciones "rand", "date_add", "uuid" y "md5" para generar los datos aleatorios
- Puede hacer uso de condicionales y ciclos en la forma que desee
- Recuerde que el procedimiento pr_fill se ejecuta una sola vez para cada tabla, y en esa ejecución debe realizar todos los insert requeridos
- Los valores de "my_value_type" deben calcularse acorde a lo generado en "my_value". Si el número es par el valor es "EVEN", de lo contrario es "ODD".
- Para las pruebas iniciales, no usar más de 10 registros en el parámetro de entrada.

EJERCICIOS

Ejercicio 3

Crear un procedimiento almacenado llamado "pr_clear", con parámetro de entrada p_table_name, varchar(30), y parámetro de salida p_records unsigned bigint.

El procedimiento debe eliminar la totalidad de la información en la tabla p_table_name y dejar en el parámetro p_records la cantidad de registros eliminados.

EJERCICIOS

Ejercicio 4

Ejecutar el procedimiento almacenado pr_fill 6 veces, una por cada tabla, con 1.000 registros. Tomar los tiempos de ejecución de cada motor de almacenamiento. Posteriormente ejecutar el procedimiento pr_clear y tomar nota del parámetro de salida.

No olvidar iniciar una transacción antes y confirmarla luego, de lo contrario la ejecución tomará horas

Repetir el ejercicio con 50.000, 100.000, 300.000 y 500.000 registros y tomar nota de los tiempos en cada motor.

EJERCICIOS

Ejercicio 5

Repetir el ejercicio 4 con 1'000.000 de registros, y si el tiempo lo permite, con 5'000.000. Tomar nuevamente nota de los tiempos en cada motor.

EJERCICIOS

Ejercicio 6

Ejecutar la consulta de la cantidad de registros con id entre 140.000 y 160.000 sobre cada una de las tablas. Tomar nota de los tiempos de respuesta en cada motor.

```
select  count(*)
from    t_xxxxx
where   id between 140000 and 160000;
```

EJERCICIOS

Ejercicio 7

Ejecutar la consulta de la cantidad de registros que son número par y luego repetirla para impares, sobre cada una de las tablas. Tomar nota de los tiempos de respuesta en cada motor.

```
select  count(*)
from    t_xxxxx
where   my_value_type = 'ODD'

// repetir la sentencia sql cambiando ODD por EVEN.
```

EJERCICIOS

Ejercicio 8

Sacar conclusiones respecto a los tiempos:

- ¿Cuál fue el motor más rápido para insertar?
- ¿Cuál fue el motor más rápido para borrar?
- ¿Cuál fue el motor más rápido para consultar?
- ¿Porqué al insertar muchos registros ocurre el error con el motor "memory"?
- ¿Porqué los tiempos se comportan de esa manera con el motor "blackhole"?

¿PREGUNTAS?

